

CIRO

Product Requirements & Build Story

Vision, personas, functional requirements, engineering decisions, impact, and roadmap

Author	Saqib Azair
Document No.	CIRO-DOC-004
Version	3.0
Date	September 01, 2026
Project	Alibaba Cloud x Bano Qabil Hackathon 2026
Status	Final Submission

This document is the Product Requirements Document (PRD) for CIRO v3.0 combined with the full engineering build story. It documents the problem we set out to solve, the personas we designed for, every functional requirement, the key engineering decisions made, measured impact, and the forward roadmap.

Website	https://ciroquick.netlify.app
API Backend	https://bano-qabil-alibabacloud-hackthon-ciro-app-production.up.railway.app
Repository	https://github.com/saqib54/Bano-Qabil-AlibabaCloud-Hackthon-CIRO-APP

This document is part of the CIRO official documentation suite prepared for the Alibaba Cloud x Bano Qabil Hackathon. All content is based on the implemented system.

01

Vision & Problem Statement

Pakistan's emergency response system is fragmented, slow, and inaccessible to much of the population. Citizens report emergencies by phone — a process hampered by language barriers, vague location descriptions, and operator overwhelm. Control rooms receive a flood of unverified, duplicate, and false reports. Responders receive raw, unverified assignments. Citizens receive no feedback after reporting. Urdu and Roman Urdu speakers are excluded by English-first systems.

Pain Point	Current Reality	Impact
Reporting is slow	Phone calls with language barriers, unclear descriptions	Delayed response times; critical first-response window lost
Noise overwhelm	Fake, spam, and duplicate reports flood control rooms	Real emergencies deprioritised; responders dispatched to false alarms
Zero citizen feedback	"Report filed" — then silence	Citizens panic, call again, create more duplicate reports
Unverified assignments	Responders receive raw data with no context or priority	Inefficient deployment; responders arrive without preparation
Language exclusion	English-first interfaces exclude Urdu/Roman Urdu speakers	Vulnerable populations unable to report effectively in emergencies

KEY POINT Our Thesis: An AI-verified, human-approved pipeline can compress report-to-dispatch from minutes to seconds while keeping final authority with humans and maintaining full transparency and accountability at every step.

02

User Personas

Persona	Profile	Core Needs	Pain Points Solved
Citizen (Ayesha, Lahore)	Non-technical, Urdu-first speaker, mobile-only access	Report in Urdu or by voice, get instant confirmation, clear instructions	Language barrier, no feedback, unclear process, no status updates
Responder (Bilal, Rescue 1122)	Field staff, duty-status based, needs clear assignments with location	Verified cases, assignments with location, clear instructions, real-time updates	Unverified reports, vague locations, no context, outdated info
Dispatcher (Saqib, Command Center)	Senior operator responsible for coordination, needs full system oversight	Efficient routing, confidence, speed of decision-making, comprehensive data	Overwhelm, duplicate reports, no context, no ability to track or verify

03

Functional Requirements — All Priorities

ID	Priority	Requirement
FR-01	P0	Emergency report submission: category, description, GPS, photo evidence, contact phone; citizen sees ov
FR-02	P0	Voice-to-Report: mic capture in ur-PK locale; live interim transcription; AI autofill of category/location/peop
FR-03	P0	AI Auto-Triage: 10-agent deterministic pipeline (Sentinel, SpamGuard, GeoScout, SatelliteScout, DedupG
FR-04	P0	Instant safety reply: authority-approved template per emergency category shown to citizen immediately —
FR-05	P0	Human approval gate: dispatch only after dispatcher clicks Approve & Dispatch; audit-logged; citizen rece
FR-06	P0	Live alerts: public alerts for verified critical/high incidents; area citizens notified; alert ticker + notification ce
FR-07	P0	Safety map: live incident pins with severity pulse; impact-zone polygons; forecast hotspot rings; shelters; m
FR-08	P1	Spam/fake detection: SpamGuard score >= 40 blocks auto-actions; >= 60 forces human review; all finding
FR-09	P1	Duplicate detection: DedupGuard matches same-area recent reports; corroborating reports raise confiden
FR-10	P1	Satellite signal fusion: SatelliteScout adjusts confidence for fire/flood/landslide; SUPPORTING +6, CONFL
FR-11	P1	Medical privacy: medical impact zones are private — teams only, zero public exposure, affected count sto
FR-12	P2	AI Decision Center: NEEDS_REVIEW / LOW_CONFIDENCE / DUPLICATE queues with confidence %, s
FR-13	P2	Emergency forecasting: hotspot grid (~2 km) from last N days; risk = min(100, count*14 + weight*6); show
FR-14	P2	Command Copilot: 2-3 line plain-language dispatcher summary per incident, stored in copilot_summary co
FR-15	P3	Design system: navy #0A1E42 surfaces, periwinkle #D7E3F8 canvas, aqua #4CC9F0 accents; curved arc
FR-16	P3	Onboarding: welcome -> consent checklist -> location -> notifications -> microphone, with permission resu
FR-17	P3	Navy sidebar across all portals: gradient logo tile, active item solid blue, particle-network art, user card wit

04

Product Specifications — Target vs. Delivered

Specification	Target	Delivered	Result
Triage latency	< 10 s	< 5 s (10-agent)	Exceeded
Languages	Urdu, English, Roman Urdu	All 3, runtime switch	Met
Auth options	Password + social + passwordless	All 3 modes	Met
Portals	Citizen, responder, command	3 portals, 34 screens	Met
Realtime	Instant alerts	WebSocket fan-out	Met
Mobile	Android + iOS	APK shipped + iOS ready	Met
API surface	REST, documented envelope	82 endpoints	Exceeded
Human oversight	Mandatory approval gate	Audit-logged dispatch	Met
Medical privacy	Medical confidentiality	Team-only zones	Met
Satellite fusion	Signal integration	FIRMS/Sentinel-1 sim	Met
Forecasting	Hotspot identification	2 km grid, risk score	Met
Client TTI	< 2 s	~1.2 s (Vite dev)	Exceeded
WebSocket latency	< 1 s	< 300 ms	Exceeded

05

Key Engineering Decisions

Why we chose these approaches and what problems they solved

Decision	Rationale	Alternative Considered
SQLite via better-sqlite3	Zero-ops single-file database; transactional; synchronous SQL	Postgres SQL eliminates fallback complexity
Deterministic 10-agent chat	Explainable decisions with per-agent trace for every input	Single QwS call for functionality
Firestore tokens verified via 50 endpoints	Local info endpoint rejects Firebase tokens	Token RS256 verification against Google Firebase
Runtime API URL via runtime config	Frontend for mobile can switch between Railway, Modal, and Uba	Static URL hardcoded in iOS binary
Capacitor 8 over native react	One React codebase produces web + Android APK	React Native requires a different permission model
Human approval gate before dispatch	For medical safety: AI recommends based on data, human makes the final dispatch decision	Full automation negates the idea of a decision gate
Deterministic fallback for every feature	Every feature-powered feature has a complete rule-based fallback	The system is fully functional without AI

06

Engineering Challenges Solved

Challenge	Solution Implemented
Firestore ID tokens rejected by phoneinfocloud RS256 verification: fetch Google securetoken x509 public key certificates (https://www.googleapis.com/auth/cloud-platform/public-keys)	Implemented local RS256 verification: fetch Google securetoken x509 public key certificates (https://www.googleapis.com/auth/cloud-platform/public-keys)
Google sign-in impossible inside Capacitor WebView using googleplus for native Android account picker. Credential result is passed as null	Used Capacitor WebView using googleplus for native Android account picker. Credential result is passed as null
API URL varies per platform and environment	env.config.json is fetched before React app initialisation. Vite env var is used as backup
Capacitor build requires Java 21	Configure Gradle SDK toolchain resolver to download and use JDK 21 automatically. Documented
Ephemeral-disk Railway deployment	By using SQLite database as persistent storage, migrations on every cold start. Conditional seed (IF use
CORS failures between web (REACT) and Android WebViews, complex vite (as multiple parts)	Backend parses the
Rate limiting OTP without blocking legitimate users	Using rate-limit instance for /auth/otp/* at 10 requests per 15 minutes per IP. Separate

07

Impact & Measured Outcomes

Stakeholder	Outcome Delivered	Measurable
Citizens	Can report an emergency in Urdu by voice in under 60 seconds	End-to-end report action time: < 60 seconds
Citizens	Receive live ETA and team details instead of silence after Websocket dispatch notification	Dispatch notification: < 300 ms from app
Citizens	Access nearby safe places and area alerts on the safety shelter directory + real-time alert ticker always available	Shelter directory + real-time alert ticker always available
Responders	Receive only verified, de-duplicated, context-rich assignments	SpamGuard + DedupGuard filters noise before assignment
Responders	Have copilot summary, impact zone, and suggested approach	Copilot and GeoImpact output on every assignment
Command Center	Process AI-scored decision queues instead of raw call logs	AI Decision Center with confidence %, satellite signal
Command Center	Pre-position resources using hotspot forecasting	2 km grid forecast from last N days of historical incidents
Command Center	Have a tamper-evident audit trail for every dispatch decision	audit_logs table: actor, action, before/after values, time

Metric	Value
AI triage latency (p95)	< 5 seconds end-to-end; 2-20 ms offline fallback
Spam auto-block threshold	Score >= 40 gates auto-actions; >= 60 forces human review
Total REST API endpoints	82 across 11 route modules
Total screens	34 across 3 portals
Supported languages	3: Urdu, English, Roman Urdu
Deployment targets shipped	Web (Netlify), API (Railway), Android APK, iOS project
Database tables	18 tables, 10 SQL migration files
Emergency categories	10 with tailored AI handling

Metric	Value
AI agents per report	10 deterministic, per-agent auditable
WebSocket push latency	< 300 ms measured

08

Forward Roadmap

Horizon	Item	Dependency
Now (v3.1)	SMTP integration with Resend or Alibaba DirectMail for real-time email delivery	SMTP credentials; no code change — mailer is pluggable
Now (v3.1)	Move API from Railway to Alibaba Cloud ECS with pm2 + nginx	ECS instance provisioned; runtime-config.json updated
Now (v3.1)	Connect Qwen DashScope for live AI inference (text + vision)	DASHSCOPE_API_KEY; fallback already in place
Next (v3.2)	SMS gateway fallback for OTP (Alibaba Cloud SMS) — for cities without Twilio	Alibaba Cloud SMS API credentials
Next (v3.2)	Firebase Cloud Messaging (FCM) push notifications for mobile apps	FCM server key; Capacitor push notification plugin
Next (v3.2)	Responder live GPS tracking — real-time location updates to map	Background location plugin for Capacitor; WebSockets
Later (v4.0)	Real satellite feed ingestion: NASA FIRMS fire hotspots and Sentinel-1 flood data	API keys and data pipeline for FIRMS and Copernicus
Later (v4.0)	ML hotspot model retraining on production incident history	Sufficient historical data; Alibaba Cloud PAI ML platform
Later (v4.0)	Multi-city federation: separate CIRO instances for Lahore, Karachi, Islamabad, etc.	Cloud infrastructure; identity federation
Later (v4.0)	Official integrations with Rescue 1122, Police 15, hospital networks	Government partnerships and data sharing agreements